

Exercise 1

Beginnings

GHC provides either a compiler, with which we can compile a Haskell program into an executable, and an interpreter with a [read-eval-print-loop](#). Initially, we'll start off using the interpreter. It can be invoked with `ghci`, so at a terminal prompt run

```
$ ghci
```

When I show commands in the terminal, I will use a `$` to indicate the prompt (which you should not type), followed by the command to type.

If everything is installed correctly, you should see a prompt (something like this)

```
$ ghci
GHCi, version 8.10.1: https://www.haskell.org/ghc/  :? for help
Prelude>
```

Hello, World!

You can type single lines of Haskell code at the prompt and the interpreter will evaluate them for you. Strings, which evaluate to themselves will just be echoed by the interpreter, so the canonical [Hello, World](#) program is

```
Prelude> "Hello, World!"
"Hello, World!"
```

When I show commands in the Haskell interpreter, I'll use `Prelude>` as the prompt, again, you shouldn't type this.

If instead, we want to print the string to the screen (so we don't have the quote marks) we can use `putStrLn`

```
Prelude> putStrLn "Hello, World!"
Hello, World!
```

Here we can make our first observation about Haskell. There are no parentheses for function calls.

Code in files

It is tedious to always type things at the prompt, and it's easy to make a mistake. Instead, we'll put things in files. These conventionally have the suffix `.hs`.

Let's go ahead and make a file `exercisel.hs` with the contents¹

```
hello :: IO ()
hello = putStrLn "Hello, World!"
```

We can make this available in the interpreter either by running

```
$ ghci exercisel.hs
GHCi, version 8.10.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main                ( exercisel.hs, interpreted )
Ok, one module loaded.
*Main>
```

Or else calling `:load exercisel.hs` after starting the interpreter

```
Prelude> :load exercisel.hs
[1 of 1] Compiling Main                ( exercisel.hs, interpreted )
Ok, one module loaded.
*Main>
```

Notice how the prompt changed, indicating that this new module which defines our functions is available.

Now, all the functions we defined in our file are available, and we can call them at the prompt

```
*Main> hello
Hello, World!
```

If you edit the file to fix bugs or define new functions, you'll either need to restart the interpreter and reloading, or using the `:reload` command.

To exit the interpreter type `Control-d`, or use the `:quit` command.

First steps

Read through the “First steps” slides (in lecture 2) from [Graham Hutton's notes](#) and try some of the expressions.

Naming requirements and layout rule <#>

Haskell has a bunch of requirements on the way types, functions, and variables are named, along with the syntax of function calls.

This code is nearly right, but not quite

```
N = a 'div' length xs
  where
    a = 10
    xs = [1, 2, 3, 4, 5]
```

Having read some of the slides, you should be able to fix it.

Some simple library functions

Now we'll reimplement some of Haskell's library functions on lists to get a feel of how to do so. So that you can check your result, I recommend naming your implementation with a trailing `'`.

Exercise 1.1

The function `last` selects the last element of a list (and produces an error on the empty list). Using the functions introduced in the slides above, write a function `last'` which does the same.

Question 1.2

Can you think of a different definition?

Exercise 1.3

The function `init` removes the last element of a list (and produces an error on the empty list). Write two different implementations, `init'` and `init''` which do the same.

Exercise 1.4

Write a function `shuffle` which takes as an argument a non-empty list (that is, you may assume that the list has at least one element), removes the first element, and appends it at the end. So, for example

```
Prelude> shuffle [1, 2, 3]
[2, 3, 1]
```

If your first attempt didn't work, why not?

Having done that, use pattern matching to extend your definition to handle empty lists, where `shuffle []` should produce `[]`.

Finally, an often-used example of the succinctness of Haskell is presented on the last slide of Hutton's first lecture. I reproduce it below (without its type annotation)

```
mystery [] = []
mystery (x:xs) = mystery ys ++ [x] ++ mystery zs
  where
    ys = [a | a <- xs, a <= x]
    zs = [a | a <- xs, a > x]
```

Question 1.5

This function clearly operates on lists, but given a list, what does it return?

Can you spot the algorithm that is used?

Finally, what would happen if you replaced the line

```
ys = [a | a <- xs, a <= x]
```

with

```
ys = [a | a <- xs, a < x]
```